# CA2: Networking + Version Control

Photon Fusion Multiplayer Feature and Source Control Evidence

## A1. Session Setup and Spawn Baseline

The Fusion NetworkRunner is configured in Host mode using the Photon Cloud relay. A BasicSpawner script manages session creation, player spawning, and a shared ScoreManager object. On player join, the server spawns a Player NetworkObject with InputAuthority assigned to the joining player. A ScoreManager NetworkObject is spawned once by the host to track shared game state.

### Spawner Configuration

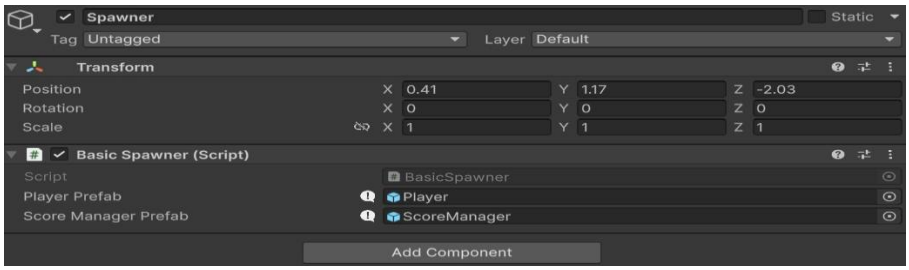


*Fig A1.1 — BasicSpawner Inspector showing Player and ScoreManager prefab references.*

### Player NetworkObject Prefab

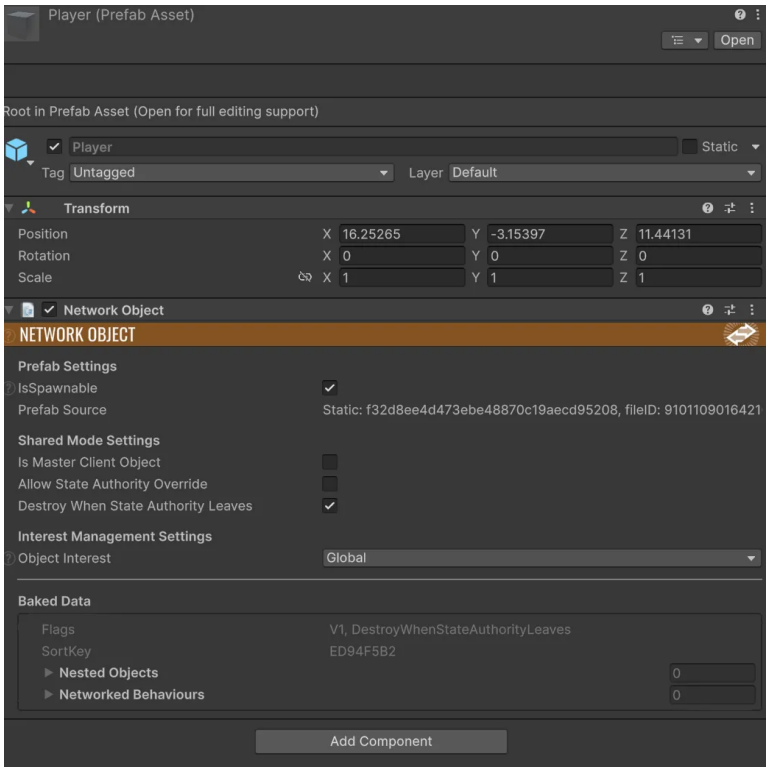


*Fig A1.2 — Player prefab with NetworkObject component. IsSpawnable enabled, Object Interest set to Global.*

## Two-Client Baseline

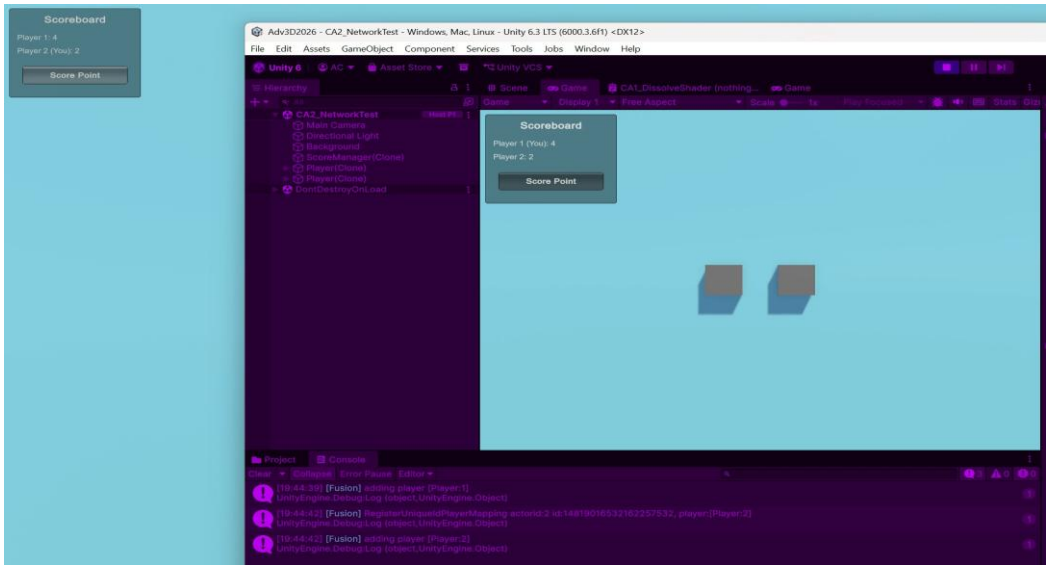


*Fig A1.3 — Two-client session: Host (right) and Client (left) both connected with synchronised scoreboard showing Player 1: 4, Player 2: 2.*

# A2. Feature Implementation: Networked Score System

## Authority Code Excerpts

The ScoreManager uses [Networked] properties for persistent state and an RPC for client-to-server score requests:

```
[Networked] public int ScorePlayer1 { get; set; }
[Networked] public int ScorePlayer2 { get; set; }

[Rpc(RpcSources.All, RpcTargets.StateAuthority)]
public void RpcRequestScore(PlayerRef player) {
    if (!Object.HasStateAuthority) return;
    if (player.RawEncoded == 2) ScorePlayer1++;
    else ScorePlayer2++;
}
```

The HasStateAuthority guard ensures only the host modifies the networked score values. Clients send score requests via the RPC but cannot write to [Networked] properties directly.

## Synchronisation Explanation

The score values are [Networked] properties because they represent persistent state visible to all clients, including late joiners. Fusion automatically synchronises these whenever the StateAuthority modifies them. The RPC serves as a request mechanism: any client calls it, but only the StateAuthority processes the change. This pattern — RPC for requests, [Networked] for state — is the standard authority-correct approach. An RPC-only approach would fail for late joiners since RPCs are fire-and-forget and not replayed.

## Test Matrix

## # Scenario->Expected Outcome->Actual result

**1. Host starts session**

**Session created, host player spawned, ScoreManager spawned**

**Session created successfully, cube visible, scoreboard displayed**

**2. Client joins session**

**Client connects, second player spawned, both cubes visible**

**Client joined, both cubes visible on both windows**

**3. Host clicks Score Point**

**ScorePlayer1 increments on both clients**

**Score updated to 1 on both windows simultaneously**

**4. Client clicks Score Point**

**ScorePlayer2 increments on both clients**

**Score updated to 1 on both windows simultaneously**

**5. Both players score multiple times**

**Scores accumulate independently and correctly**

**P1: 3, P2: 5 displayScores retained on host, client sees current scores on rejoined correctly on both clients**

**6. Client disconnects and rejoins**

**Scores persist (maintained by host StateAuthority)**

**Scores retained on host, client sees current scores on rejoin**

**7. "(You)" label displays correctly**

**Host sees "(You)" next to Player 1, client sees it next to Player 2**

**Labels display correctly on each respective client**

## A3. Unreal Replication Terminology

**Authority Roles**. Fusion has two authority roles: StateAuthority, which controls who can modify an object's networked state, and InputAuthority, which controls who provides input for it. In Unreal, the Server is essentially the StateAuthority, it always holds the truth and is the only thing that can change replicated state. The Owning Client is the closest thing to InputAuthority,  it's the client tied to a PlayerController that sends input, but can't directly change state. Unreal also has Simulated Proxies, which are basically how objects look on clients that don't own them.

**[Networked] vs Replicated UPROPERTY**. Fusion's [Networked] attribute and Unreal's UPROPERTY(Replicated) do essentially the same thing, they mark a variable to be automatically synced from the authority to all clients. Both handle late joiners, so if someone connects mid-game, they get the current values straight away. Unreal has an extra feature called ReplicatedUsing which fires a callback (OnRep) on clients when the value changes, which is handy for triggering UI updates. Fusion can do something similar with OnChanged callbacks but it's a less built-in.

**RPCs**. Fusion lets you specify who can send an RPC and who receives it (ProSource's and RpcTargets). Unreal instead has three fixed types: Server RPCs go from client to server, Client RPCs go from server to a specific client, and NetMulticast goes from server to everyone.

**Key Difference**. The thing that stood out to me most was how authority works with shared objects. In Unreal, the server owns everything, there's no built-in way for a client to be authoritative over an object. Fusion has a Shared mode where different clients can hold StateAuthority over different objects.

## B. Version Control Evidence

### Commit Graph and Branch Structure

```
*   a2ea2b3 (HEAD -> master) merge: feature/ca2-network into master
|\
| * ca7e194 (feature/ca2-network) docs: learning journal and unreal replication note
|/
* 3242be9 (origin/master) net: networked score system with polished UI
* f0f28b3 docs: CA2 plan - scope decision
* af09b1b (tag: ca2-baseline) net: fusion session setup - two client baseline working
* 4bb8d49 (tag: ca1-submit) feat: CA1 complete - dissolve shader, dusk lighting, post-processing
```

*Fig B.1 — Commit graph showing tags (ca1-submit, ca2-baseline), feature/ca2-network branch, and merge to master.*

The graph shows: CA1 tagged at ca1-submit, networking baseline at ca2-baseline, plan and score system on master, then documentation on feature/ca2-network merged back with --no-ff to preserve branch visibility.

### Branch Strategy

master is kept stable. feature/ca2-network was created for the networking documentation sprint, keeping work separate. Merged back with --no-ff. Commit prefixes: net: for networking, feat: for features, docs: for documentation, fix: for bugs.

# Learning Journal

## CA2 Learning Journal

### Section 1 — Weekly Evidence Log

| 8 | 16–22 Mar | Fusion session setup; NetworkRunner configured; Player prefab spawning on two clients; CA2 scope decided (Option C — score/game state) | net: fusion session setup | Fig TBD(two-client baseline) | Photon player IDs start at 2, not 1 — had to adjust score attribution logic |
| 9 | 23–29 Mar | Delayed week, no commits
| 10 | 30–5 Mar-Apr | Score system: Networked properties, RPC with authority check, polished UI; test matrix; docs | net: networked score system, docs: CA2 plan, docs: test matrix | Fig TBD (score system, both clients) | RPC initially used RpcSources.InputAuthority on a shared object with no input authority — changed to RpcSources.All |

## Section 2 — Critical Discussion

### Why I Chose Option C (Score / Game State)

I chose a networked score system because it is a clearly scoped feature that could be fully implemented and tested within the networking sprint, and could easily serve as the basis for any other feature or be more appliccably imported to use as a baseline template in future projects. A smaller, fully correct feature was a better use of the time than a larger one that had less direct emphasis on the networking features being studied.

### Networked Property vs RPC

The score system uses both Networked properties and an RPC, each for a specific reason. The two score values (ScorePlayer1, ScorePlayer2) are stored as Networked int properties because they represent persistent game state that every client needs to see at all times. Fusion automatically synchronises Networked properties to all clients, including late joiners, if a new client connects mid-game, they immediately receive the current scores without any extra logic.

The RPC (RpcRequestScore) is used as a request mechanism: when a client clicks "Score Point", the RPC sends a message to the StateAuthority asking it to increment the score.

### A Model Decision

When I first implemented the ScoreManager, I attached it to individual player prefabs and used RpcSources.InputAuthority to send score requests. This failed because the ScoreManager needed to be a shared object (one instance tracking both players' scores), and a shared object has no InputAuthority assigned to any specific player. The RPC silently never fired from the client because RpcSources.InputAuthority requires the sender to have input authority over the object. Moving to RpcSources.All and spawning the ScoreManager as a standalone NetworkObject (not owned by any player) fixed the issue. The StateAuthority guard inside the RPC ensures only the host processes the score change, preventing double-counting.

### One Thing I Would Do Differently

I would set up my Git repository structure correctly from day one. I initially committed CA2 work into the wrong repository (the overarching folder the project is in) and had to rebuild my commit history in the correct project folder. If I were starting over, I would verify which. git directory I was committing to before making any commits, and I would establish the feature branch immediately rather than committing networking code directly to master. I also did not commit during the second week and committed very late into the extension granted in the third week as I was putting off the project weekly labs to work on assignments.

### Section 3 — Lessons Learned

1. **Photon Fusion player IDs do not start at 1.** I assumed PlayerRef.RawEncoded would be 1 for the host and 2 for the client, but they are actually 2 and 3. Hardcoding player identification based on assumed IDs caused both players to score for the same team. Adding debug logging revealed the actual values and the fix was straightforward, but the assumption cost debugging time.

2. **Shared NetworkObjects need RpcSources.All, not RpcSources.InputAuthority.** When a NetworkObject is not owned by a specific player (like a shared ScoreManager), no client has InputAuthority over it. Using RpcSources.InputAuthority meant RPCs from non-host clients were silently dropped. Changing to RpcSources.All and relying on the HasStateAuthority guard inside the RPC resolved the issue.

3. **Always verify which Git repository you are committing to.** Having nested .git directories (one in the project root, one in a subfolder) led to commits going to the wrong repository. This was especially painful because it meant that when I did finally push to the right GitHub, I had lost a "in between" step in my implementation progress. Simply doubly checking my repo would have caught this.

## Attribution

All primary assets (Jade Toad model, PBR textures) provided through the module GitHub repository. Noise texture (Samples_FractalNoises) from Unity SRP Core samples. Photon Fusion 2 SDK under free educational tier. No additional third-party assets.